

Umgang mit dem Kursmaterial

- Sämtliches Kursmaterial (Texte, Folien, Abbildungen, Aufgaben, Lösungen usw.) unterliegt dem Urheberrecht des Dozenten und/oder weiterer Rechteinhaber.
- Die Nutzung ist ausschließlich den Studierenden der angegebenen Fachhochschule im Rahmen des angegebenen Kurses und nur zu Ausbildungszwecken gestattet.
- Jede weitergehende Nutzung, insbesondere das Veröffentlichen, Vervielfältigen, Verbreiten, Teilen oder öffentliche Zugänglichmachen des Materials (z.B. auf Webseiten, in Cloud-Speichern, per E-Mail, in Präsentationen oder Publikationen), ist ohne ausdrückliche schriftliche Zustimmung untersagt und kann rechtliche Konsequenzen nach sich ziehen.
- Die Nutzung des Kursmaterials in Verbindung mit Sprachmodellen oder KI-Systemen (z.B. LLMs wie ChatGPT) ist nur dann zulässig, wenn sichergestellt ist, dass die Inhalte nicht zum Training, zur Speicherung oder zu einer sonstigen Weiterverarbeitung durch Dritte verwendet werden (wie z.B. dem LLM-Anbieter) und alle oben genannten Nutzungsbeschränkungen eingehalten werden.



Kapitel_05

Linux 2: Prozesse und Benutzer

Betriebssysteme

Prof. Dr. Claus Fühner

Fakultät Informatik

Ostfalia Hochschule für angewandte Wissenschaften

Braunschweig/Wolfenbüttel

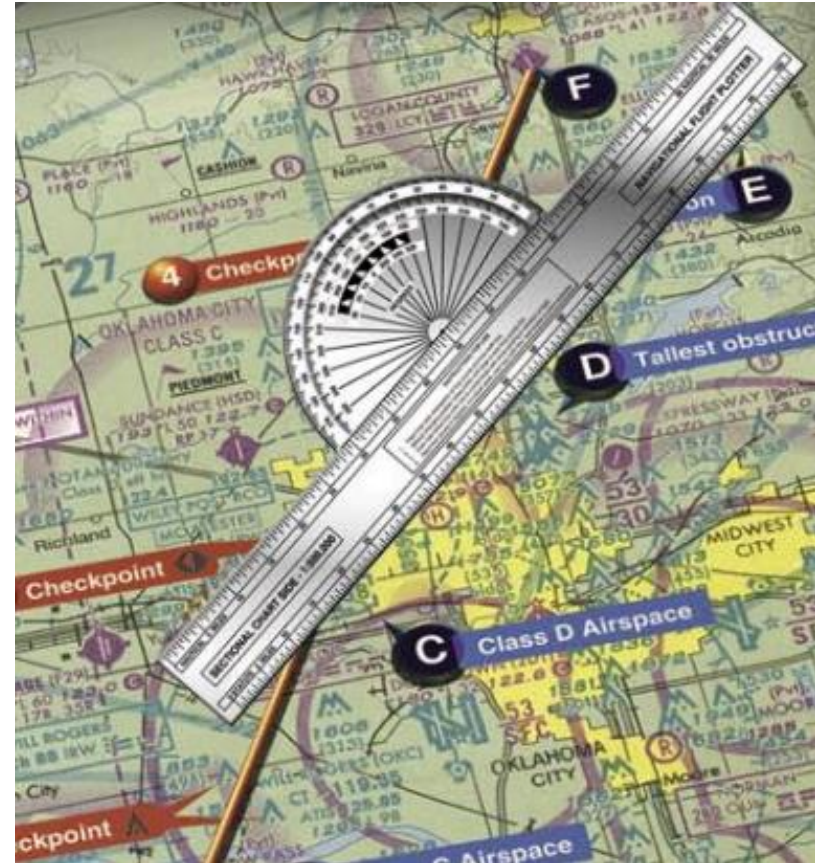
Agenda



Linux 2: Prozesse und Benutzer

Ein-/Ausgabeströme

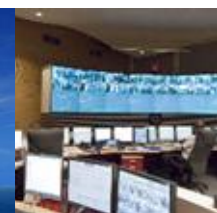
- Benutzerverwaltung
- Dateizugriffsrechte
- Hard- und Softlinks
- Prozesse und Signale
- Arbeit auf entfernten Rechnern



Nur zu Lehrzwecken



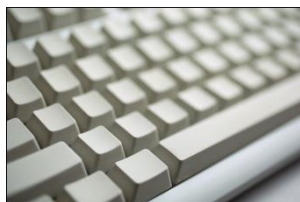
Betriebssysteme



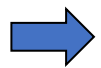


Konzept

- Unix führt das Konzept der **Ein- und Ausgabeströme** ein
→ Programme können damit von konkreten Geräten abstrahieren
- Die meisten Programme verwenden mindestens drei Ein-/Ausgabeströme
 - **Standardeingabe** (**STDIN**, 0) – hieraus werden die verarbeitenden Informationen gelesen (*Java: `System.in.read()`*)
 - **Standardausgabe** (**STDOUT**, 1) – hierhin werden die verarbeiteten Informationen geschrieben (*Java: `System.out.print()`*)
 - **Standardfehlerausgaben** (**STDERR**, 2) – hierhin werden Fehlermeldungen geschrieben (*Java: `System.err.print()`*)
- Die Shell weist einem Programm beim Start die **Tastatur** als Standardeingabe und den **Bildschirm** als Standardausgabe und Standardfehlerausgabe zu.



Standard-
eingabe



Programm



Standardausgabe

Standardfehler-
ausgabe



Umlenkung der Ein-/Ausgabe

- Die **Shell** erlaubt es, die drei Ein-/Ausgabeströme *auf Dateien* umzuleiten
- Umleitung der **Standardeingabe** mit **<**: `befehl < datei`
- Umleitung der **Standardausgabe**:
 - bisherigen Inhalt überschrieben mit **>** `befehl > datei`
 - neuen Inhalt anhängen mit **>>** `befehl >> datei`
- Umleitung der **Standardfehlerausgabe** mit **2>** `befehl 2> datei`
- Beispiel: `wc < /etc/passwd`
- Standardfehlerausgabe auf Standardausgabe umleiten: `wc 2>&1`
- Umleitungen können auch programmgesteuert erzeugt werden mit den Systemaufrufen `dup()` und `dup2()`. So realisiert auch die Shell die Umleitung.

Verkettung von Kommandos - Pipes

- Mittels einer **Pipe** können **zwei Kommandos** *unidirektional* verbunden werden:
 - Die Standard**ausgabe** des ersten Kommandos wird auf
 - die Standard**eingabe** des zweiten Kommandos gelenkt.
- Die Shell ermöglicht die Verkettung mit dem Pipe-Operator (|)
- Auf diese Weise können komplexe Befehle aus einfachen Befehlen zusammengebaut werden, man spricht auch von **Filterketten**
- Beispiel `ps -ax | sort --key=5 | less`
- Umleitungen können auch programmgesteuert erzeugt werden mit dem Systemaufruf `pipe()`. Dies ist nützlich zum Beispiel für die Kommunikation verschiedener Prozesse. So realisiert auch die Shell die Umleitung.

Programme zur Textverarbeitung

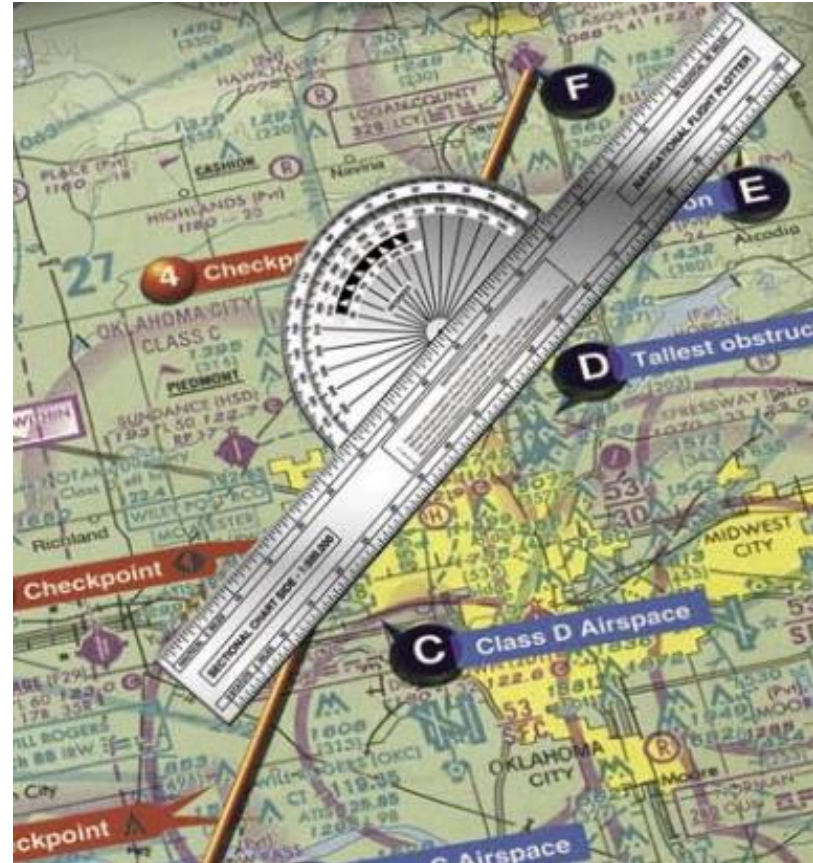
<code>cat</code>	<i>concatenate, Inhalt von Dateien ausgeben</i>
<code>cut</code>	Ausschneiden von Spalten aus Texten
<code>grep</code>	<i>get regular expression print</i> , Suchen nach regulären Ausdrücken heute nur <code>grep [-v] suchbegriff filename(s)</code> , später mehr
<code>egrep</code>	<i>extended grep</i> , Suchen nach erweiterten regulären Ausdrücken (später)
<code>head</code>	Ausgeben der ersten Zeilen einer Datei
<code>nl</code>	<i>number lines, Zeilennummerierung</i>
<code>sort</code>	Sortieren von Texten
<code>sed</code>	Stream Editor, Text skriptgesteuert modifizieren (evtl. später mehr)
<code>tail</code>	Ausgeben der letzten Zeilen einer Datei (für logs nützlich: <code>tail -f</code>)
<code>tee</code>	Aufspaltung der Standardeingabe in Standardausgabe und Datei
<code>tr</code>	<i>translate, Umformen von Texten</i>
<code>uniq</code>	Löschen doppelter Zeilen (aber Anforderung an Eingabedaten!)
<code>wc</code>	<i>word count, Zählen von Zeilen, Wörtern und Zeichen einer Datei</i>

Agenda



Linux 2: Prozesse und Benutzer

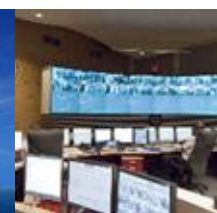
- Ein-/Ausgabeströme
- Benutzerverwaltung
- Dateizugriffsrechte
- Hard- und Softlinks
- Prozesse und Signale
- Arbeit auf entfernten Rechnern



Nur zu Lehrzwecken



Betriebssysteme



Konzept Nutzer (user)

- Unix verwendet das Konzept der **Benutzer (User)**, um separate Umgebungen zu schaffen und Rechte zu managen.
 - **root** – Superuser mit umfassenden Rechten (Prompt meist # statt \$)
 - **Standardbenutzer** - für die interaktive Arbeit
 - **Systembenutzer** - für Systemdienste wie Email, kein Login, keine Shell
- Jeder Benutzer besitzt individuell
 - **Heimatverzeichnis** (Home Directory) – hier können in versteckten Dateien viele Einstellungen gemacht werden (z.B. **.bashrc/.zshrc**)
 - **Umgebungsvariablen** (→ pro Prozess, später)
- Jeder Benutzer wird mit einer **User ID** identifiziert
- Für das System ist die **User ID** und nicht der Nutzernamen entscheidend!

Konzept Gruppe (group)

- Jeder Benutzer ist einer **Hauptgruppe (primary group)** zugeordnet
 - zudem kann man Mitglied weiterer Gruppen (**supplementary groups**) sein
 - Gruppenzugehörigkeit wichtig für Zugriff auf Dateien, Geräte und Dienste
- Typische Systemgruppen:
 - **sudo** für Benutzer, die sich mit dem Kommando **sudo** Superuser-Rechte verschaffen dürfen
 - **audio**, **video** für Benutzer, die auf entsprechende Geräte zugreifen dürfen
 - **docker** für Benutzer, die Docker (ohne sudo) nutzen dürfen
- Nutzer zu einer Gruppe hinzufügen (erst nach nächstem Login wirksam):
usermod -aG gruppe benutzer
- Jede Gruppe wird mit einer **Group ID** identifiziert.
- Für das System ist die Group ID und nicht der Gruppenname entscheidend!

Superuser auf Zeit: sudo

- **Minimalprinzip:** Einloggen oder dauerhaftes Arbeiten als Superuser vermeiden!
- **sudo** (substitute user do) – Befehle mit den Rechten eines anderen Benutzers (meist **root**) ausführen, ohne sich als **root** einzuloggen
- Voraussetzung: Mitgliedschaft in der Gruppe **sudo** (Ubuntu/Debian) oder Eintrag in **/etc/sudoers**
- Syntax:
 - **sudo befehl** – Befehl als root ausführen
 - **sudo -u benutzername befehl** – als anderer Benutzer ausführen
 - **sudo -i** – interaktive root-Shell (analog zu **su -**, aber mit eigenem Passwort)
- Vorteil gegenüber direktem root-Login: Aktionen werden geloggt (**/var/log/auth.log**) – Nachvollziehbarkeit und Minimalprinzip
- Konfiguration in **/etc/sudoers**: legt fest, wer welche Befehle ausführen darf – Bearbeitung nur mit **visudo** (Syntaxprüfung!)

Dateien und Befehle

Dateien:

- `/etc/passwd` – diese Datei enthält die Infos zu allen Benutzern
- `/etc/shadow` – enthält (gesalzene) Hashes der Passwörter
- `/etc/group` – enthält die Infos zu allen Gruppen

Befehle:

- `id` liefert User ID (UID) und Group ID (GID) des Benutzers
- `users` aktive Benutzer ausgeben
- `groups` liefert die Gruppenzugehörigkeit
- `whoami` eigener Username
- `who` eingeloggte Benutzer

Passwörter

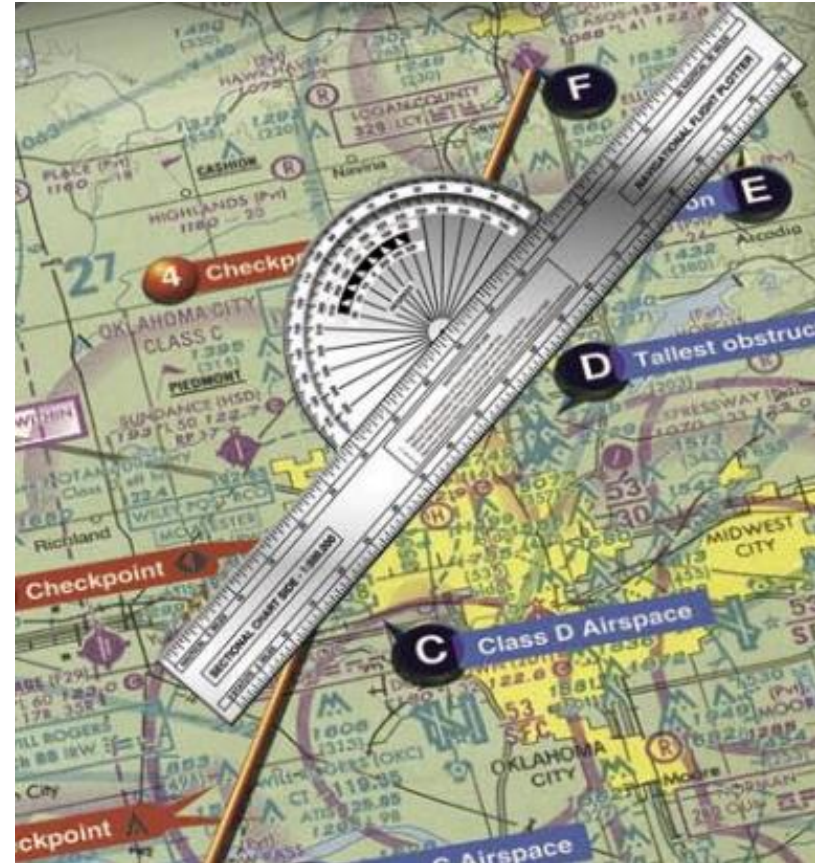
- Ändern des eigenen Passworts mit `passwd`
- Superuser kann auch Passwörter anderer Nutzer ändern, `passwd username`
- Das Passwort wird nicht im Klartext gespeichert, sondern als **gesalzener Hash**
 - **Salt**: zufälliger Wert, der mit dem Passwort gespeichert wird
 - **(Kryptographischer) Hash** wird **über den Salt und das Klartextpasswort** gebildet
 - **Dasselbe Passwort erzeugt so viele verschiedene Hashes!**
 - Schutz vor Angriffen mit Rainbow Tables
 - Beispieleintrag in `/etc/shadow`: `fuehner:$6$salt$hash`
- Nur Passwörter zu verwenden gilt als unsicher
 - Public Key Authentifizierung (über `ssh`, später, Passwörter abschalten)
 - Zweifaktor Authentifizierung

Agenda



Linux 2: Prozesse und Benutzer

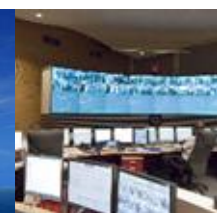
- Ein-/Ausgabeströme
- Benutzerverwaltung
- ➔ Dateizugriffsrechte
- Hard- und Softlinks
- Prozesse und Signale
- Arbeit auf entfernten Rechnern



Nur zu Lehrzwecken



Betriebssysteme



Konzept

- Jeder **Datei** ist *genau ein* Benutzer als **Besitzer (owner)** und *genau eine* **Gruppe (group)** zugeordnet.
- Diese Zuordnung einer Datei zeigt **ls -l**
- Zugriffsrechte werden für drei Benutzerklassen unterschieden
 - **User** – Rechte des Besitzers der Datei
 - **Group** – Rechte der („besitzenden“) Gruppe
 - **Others** - alle anderen Benutzer (restliche Welt)
- Mögliche Rechte sind:
 - Read **r** - Lesen
 - Write **w** - Schreiben
 - Execute **x** - Ausführen
- Alternatives Konzept für feingranulare Zugriffskontrolle mit mehreren Benutzern/Gruppen pro Datei: **Access Control Lists (ACLs)**

Wirkung auf Dateitypen

- Wirkung auf Dateien (keine Verzeichnisse):
 - Zugriffsrechte beziehen sich auf den *Inhalt* einer Datei
 - **r**: die Datei ist lesbar
 - **w**: die Datei ist veränderbar
 - **x**: die Datei ist ausführbar, d.h. ausführbare binäre Programmdatei oder ein Skript

Angabe des Skript-Interpreters mit *Shebang* in erster Skriptzeile, z.B.
`#!/bin/bash` oder `#!/usr/bin/env python3`
 - **w** impliziert kein **r**
- Wirkung auf Verzeichnisse:
 - Zugriffsrechte beziehen sich auf enthaltende Dateien und Unterverzeichnisse
 - **r**: der Verzeichnisinhalt kann aufgelistet werden
 - **w**: Dateien und Verzeichnisse können angelegt und gelöscht werden
 - **x**: in das Verzeichnis kann mit `cd` gewechselt werden
 - **r** und **x** werden typischerweise beide gesetzt

Zugriffsrechte oktal oder symbolisch

- Alternative 1: Angabe von Zugriffsrechten als **dreistellige Oktalzahl**

```
rwX  rwX  rwX
|||  |||  |||
|||  |||  |||
|||  |||  +++--- others (die Anderen)
|||  |||  421
|||  |||
|||  +++----- group (Gruppe)
|||  421
|||
+++----- user (Besitzer)
421
```

- Alternative 2: **Symbolische** Angabe von Zugriffsrechten

who**operator****permission**

↑ permission: **r** (read), **w** (write), **x** (execute)
↑ operator: **+** (hinzufügen), **-** (entfernen), **=** (gleichsetzen)
↑ who: **u** (user), **g** (group), **o** (other), oder **a** (all)

Weitere Dateiattribute

- Set User ID Bit (SUID-Bit):
 - Bei Programmen gebräuchlich: Programm wird mit den Rechten des *Dateibesitzers* (nicht des Aufrufers!) ausgeführt (gefährlich!)
 - Oktalwert: 4000, symbolisch s (bei user)
- Set Group ID Bit (SGID-Bit):
 - Programm wird mit den Rechten der Gruppe ausgeführt
 - Verzeichnis: Datei wird mit der Gruppe des Verzeichnisses erstellt
 - Oktalwert: 2000, symbolisch s (bei group)
- Sticky Bit:
 - Programm: bleibt nach der Terminierung im Speicher (historisch, heutzutage nicht mehr genutzt)
 - Verzeichnis: nur der Eigentümer einer Datei darf diese löschen oder umbenennen (**sehr nützlich!**)
 - Oktalwert: 1000, symbolisch t bzw. T

Befehle zu Zugriffsrechten

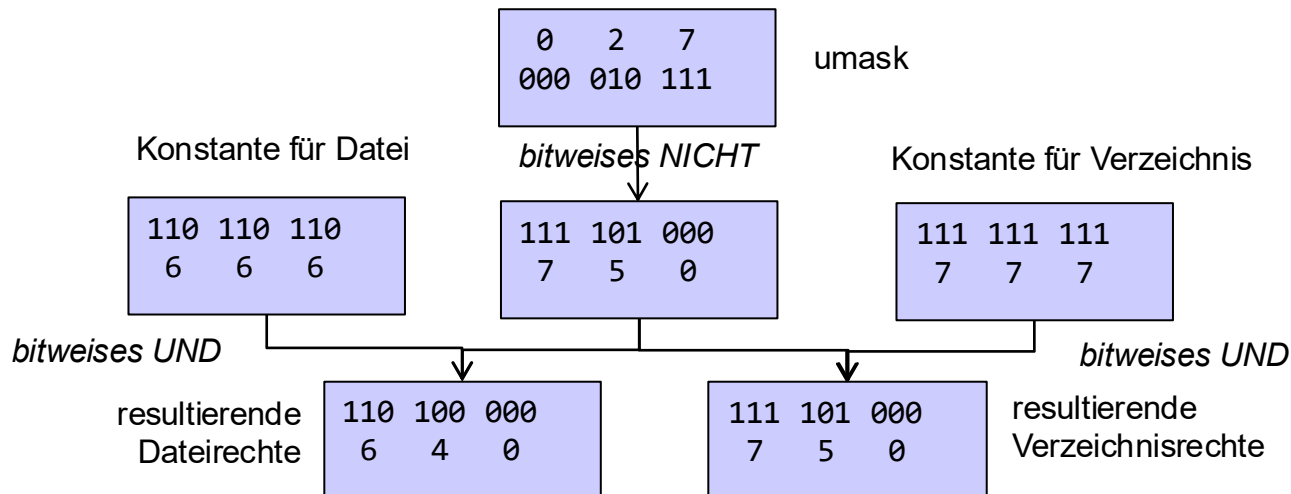
chmod Zugriffsrechte der Datei ändern

chown Besitzer der Datei ändern

chgrp Gruppe der Datei ändern

umask Maske mit Zugriffsrechten setzen

Diese Maske kommt beim Anlegen von neuen Dateien/Verzeichnissen zum Einsatz

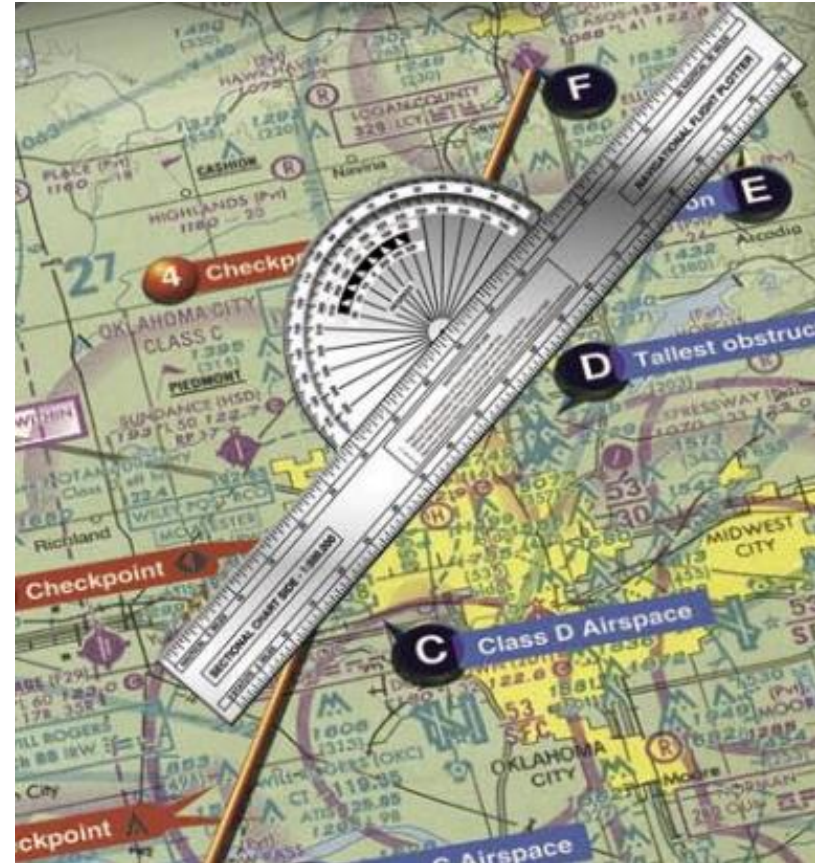


Agenda



Linux 2: Prozesse und Benutzer

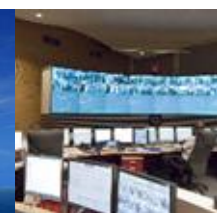
- Ein-/Ausgabeströme
- Benutzerverwaltung
- Dateizugriffsrechte
- ➔ Hard- und Softlinks
- Prozesse und Signale
- Arbeit auf entfernten Rechnern



Nur zu Lehrzwecken



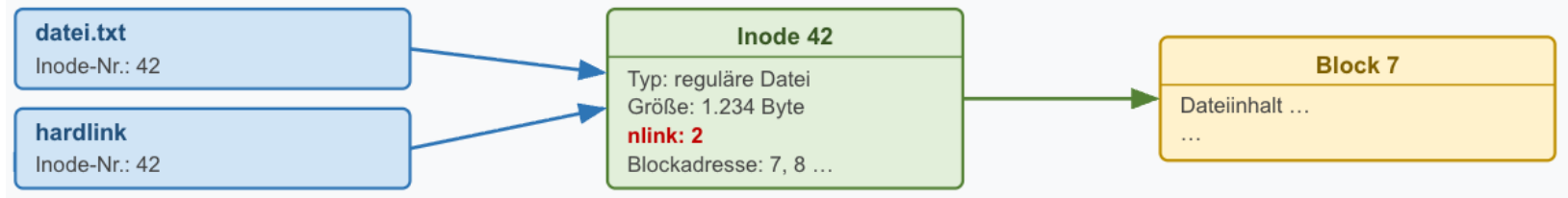
Betriebssysteme





Hardlinks

- Jede Datei hat eine **I-Node** mit Metadaten (Größe, Rechte, Eigentümer, Datenblöcke) und Verweisen auf den Inhalt
- Verzeichniseinträge verweisen nur auf Inodes
- Hardlinks sind **weitere Verzeichniseinträge**, der auf **denselben I-Node** zeigen

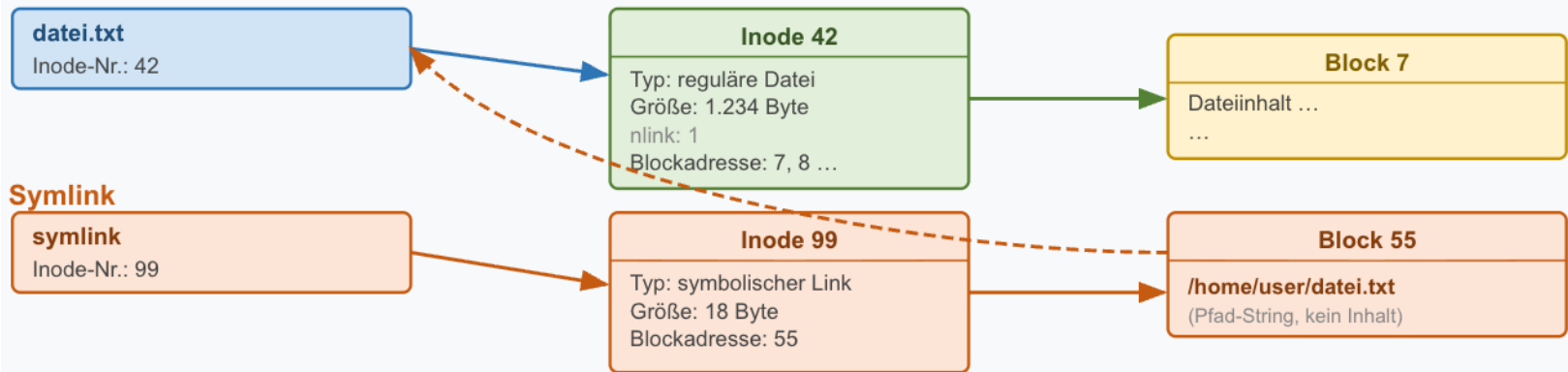


- Erstellen: `ln quelledatei linkname`
- I-Node und Link-Zähler anzeigen: `ls -li` (erste + dritte Spalte)
- Eigenschaften:
 - Gleichwertiger Zugriff: gleicher Inhalt, gleiche Rechte, gleicher Eigentümer
 - **Link-Zähler** der Inode wird erhöht → Datei existiert solange Zähler > 0
 - Löschen einer Datei (`rm`) verringert nur den Link-Zähler
 - Nur innerhalb des gleichen Dateisystems möglich (Stichwort: `mount`)



Symbolische Links

- Symbolischer Link (Symlink, Softlink): Datei, die einen **Pfad** als Inhalt enthält



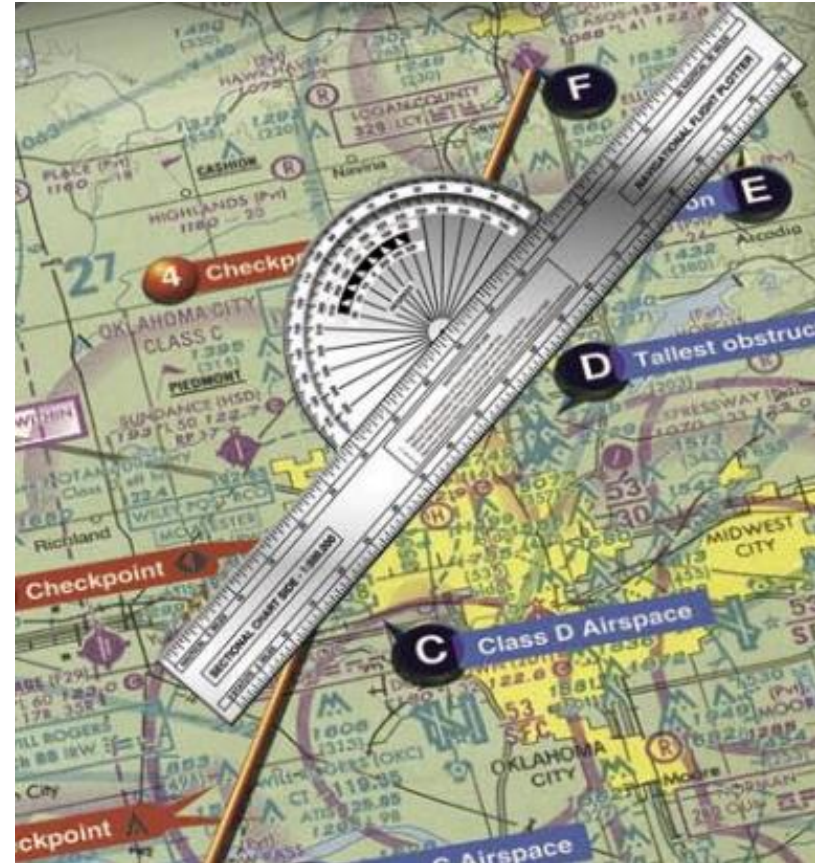
- Erstellen: `ln -s Ziel Linkname`
- Anzeigen mit `ls -l`
- Eigenschaften:
 - Berechtigungen des Links selbst irrelevant – es gelten die des Ziels
 - Zeigt auf **abs. oder rel. Pfad**, nicht auf Inode: kleine Datei mit eigenem I-Node
 - Kann auf Verzeichnisse zeigen (Hardlinks nicht ohne Weiteres)
 - Kann Dateisystemgrenzen überschreiten (auch auf Netzlaufwerke)
 - **Dangling Link**: Ziel wurde gelöscht/verschoben, **Link zeigt ins Leere**

Agenda



Linux 2: Prozesse und Benutzer

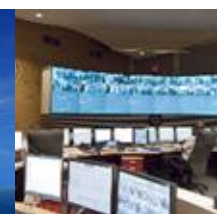
- Ein-/Ausgabeströme
- Benutzerverwaltung
- Dateizugriffsrechte
- Hard- und Softlinks
- ➔ Prozesse und Signale
- Arbeit auf entfernten Rechnern



Nur zu Lehrzwecken



Betriebssysteme



Konzept

- Jedes Kommando, das in der Shell abgesetzt wird (außer einem internen Shellkommando) führt dazu, dass ein separater **Prozess** gestartet wird.
 - 1:N Relation zwischen Programmen und Prozessen!
- Prozesse haben eine **Identität**, die **Prozess-Identifikation (PID)**.
- Wird ein Programm mehrfach gestartet, erhält jede Instanz eine andere PID.
- Prozesse können andere Prozesse starten
 - **Prozesshierarchie** mit **Eltern-, Kind-, Enkelprozessen** usw.
- Prozesse haben bestimmte Rechte
 - üblicherweise die des Benutzers, der sie gestartet hat
- Prozesse haben eigene Umgebungsvariablen (später)
- Prozesse laufen isoliert voneinander (später genauer)
- **Interprozesskommunikation**: Prozesse können Daten austauschen, zum Beispiel über eine **Pipe**

Prozesse auflisten `ps [optionen] [pid]`

- Prozessauflistung z.B. mit **ps -e** (every), zeigt alle aktiven Prozesse
- Die Ausgabe zeigt die PID, das zugeordnete Terminal (bei Systemprozessen oft nicht vorhanden), die Laufzeit und das Kommando (Prozess)

PID	TTY	TIME	CMD
1	?	00:00:09	systemd
1046888	pts/0	00:00:00	bash

- Weitere Optionen
 - a *all* alle Prozesse, die an ein Terminal gebunden sind
 - l *long* Langformat
 - f *full* vollständige Ausgabe
 - F *FULL* noch vollständigere Ausgabe
 - x auch Prozesse, die nicht an ein Terminal gebunden sind, ausgeben
- **top** oder **htop** zeigen laufend aktualisierte Liste (beenden mit q)
- **pstree** stellt Prozesshierarchie baumförmig dar

Jobverwaltung in der Shell

- Ein Prozess mit längerer Laufzeit (z.B. eine Suche) blockiert die weitere Arbeit in der Shell.
- Die Shell läuft zwar weiter, wartet aber als Elternprozess auf die Beendigung des Kindprozesses (später: Systemaufrufe `wait()` oder `waitpid()`).
- Um dies zu vermeiden, kann ein Prozess im Hintergrund gestartet werden (&):
`gcc largeprogramm.c &`
- Danach erscheint sofort ein neues Prompt zur Weiterarbeit in der Shell. Die Konsolenausgaben mischen sich.
- `Strg-Z` hält den aktuellen Vordergrundprozess an zur Weiterarbeit in der Shell
- `bg` lässt den angehaltenen Prozess (*Job*) als Hintergrundprozess weiterlaufen
- `fg` holt angehaltenen Prozess (*Job*) wieder in den Vordergrund bringen und setzt diesen fort
- `jobs` zeigt eine Übersicht der Jobs, die aus dieser Shell gestartet wurden

Nützliche Utilities um Jobs und Prozesse

- `nohup cmd` führt `cmd` im Hintergrund aus ohne Bindung an das Terminal; läuft bei Logout weiter
- `nice cmd` führt das Kommando mit geänderter Priorität für Prozess aus
- `time cmd` führt `cmd` aus und gibt Laufzeit/Ressourcennutzung aus
- `watch [-n seconds] cmd` führt das Kommando `cmd` periodisch aus
- `pgrep attribute` sucht Prozesse z.B. nach Namen und gibt deren PID aus
- `sleep seconds` erzeugt einen Prozess, der `seconds` Sekunden lang schläft
- `tmux` und der Vorgänger `screen` sind als Terminal Multiplexer eine leistungsfähige Alternative z.B. zur Verwendung von `nohup`

Signale

- Signale dienen zur Kommunikation zwischen Prozessen
- Ein Signal bewirkt beim Empfänger eine **Unterbrechung**
- Der Prozess führt nach Empfangen eines Signals eine der nachstehenden Signalbehandlungen aus:
 - Ignorieren (**IGN**)
 - **Terminierung** des Prozesses (**KILL**)
 - Terminierung des Prozesses mit **Core Dump** (**CORE**)
 - **Stoppen** des Prozesses (**STOP**, wie **Strg-Z**)
 - **Aufruf eines Signalhandlers** (normale Funktion z.B. in C)
(später: Signalhandler registrieren mit Systemaufruf `signal()`)
- Nach der Signalbehandlung läuft der Prozess an der unterbrochenen Stelle weiter (außer bei Terminierung)
- Konzept ähnelt Interrupts (ist aber etwas anderes!)

Signale

- Zum Senden eines Signals dient das Kommando (Default-Signal: 15/TERM)

`kill -<Signal> PID`

Signal	Nummer	Erklärung
SIGTERM	15	Prozess bitten, sich zu beenden (-> Signalhandler)
SIGINT	2	ähnlich SIGTERM, in der Shell durch CTRL-C ausgelöst
SIGKILL	9	Beenden eines Prozesses erzwingen, ohne dass der Prozess Aufräumarbeiten vornehmen kann
SIGSTOP	19	Prozess anhalten, kann später durch SIGCONT fortgesetzt werden; in der Shell durch CTRL-Z ausgelöst
SIGCONT	18	Prozess fortsetzen

Beispiele:

`kill -INT 4711`

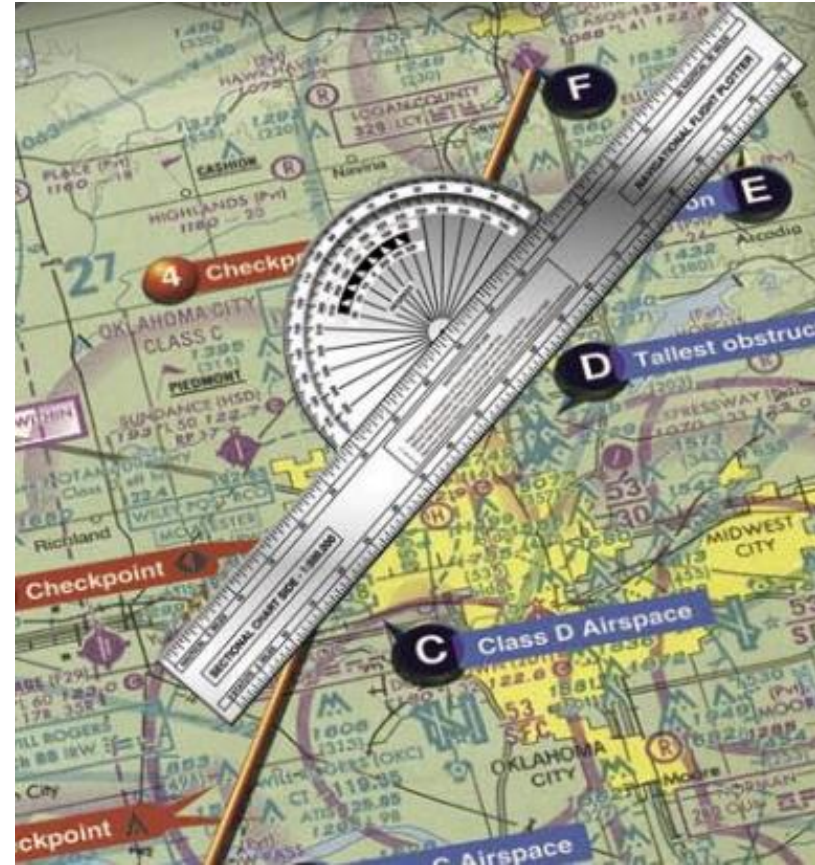
`kill -9 4711`

Agenda



Linux 2: Prozesse und Benutzer

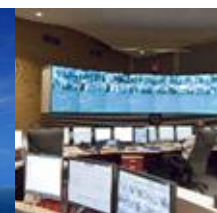
- Ein-/Ausgabeströme
- Benutzerverwaltung
- Dateizugriffsrechte
- Hard- und Softlinks
- Prozesse und Signale
- Arbeit auf entfernten Rechnern



Nur zu Lehrzwecken



Betriebssysteme



Symbolischer Mode von chmod

chmod <who> <operator> <permissions>

<who> - Wessen Rechte sollen modifiziert werden?

- u,g,o,a für Eigentümer (user), Gruppe (group), Andere (other), alle (all)
- auch Kombinationen wie ug möglich
- fehlt die Angabe von who, wird a angenommen

<operator> - wie sollen die Rechte modifiziert werden?

- +,-,= für hinzufügen, entfernen, setzen/überschreiben

<permissions> - welche Rechte sollen modifiziert werden

- r,w,x für lesen, schreiben, ausführen
- s für SUID (Set User ID, wenn u=s) oder SGID (Set Group ID, wenn g=s)
- t für Sticky Bit

Ausgabe `ls -l`:

- `s/t` erscheinen anstelle von `x`
- Grossbuchstabe für `S/T`, wenn `x` nicht gesetzt; sonst Kleinbuchstabe `s/t`

Terminal-Steuerung

- Strg-H oder Backspace: Zeichen löschen
- Strg-D : EOF Dateiende, Eingabetext zu Ende
- Strg-S : Stop Bildschirmausgabe anhalten
- Strg-Q : Start Bildschirmausgabe fortsetzen
- Strg-C : Programmabbruch
- Strg-L : Bildschirm löschen
- Strg-Z : Jobs/Prozess stoppen (NICHT beenden, siehe jobs/fg/bg)